

Correction détaillée — La boucle « while » (recueil)

Exercice 1 — Suite de Syracuse

► **1. Fonction complétée.** Si n est pair ($n\%2 == 0$), on le divise par 2 (avec `//` pour garder un entier) ; sinon, on calcule $3n + 1$.

```
1 def syracuse(n):
2     while n != 1:
3         if n%2 == 0:
4             n = n//2
5             print(n)
6         else:
7             n = 3*n+1
8             print(n)
```

► **2. Pour $n = 6$,** la console affiche successivement :

```
3    10    5    16    8    4    2    1
```

($6 \rightarrow 3 \rightarrow 10 \rightarrow 5 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$.)

► **3. Conjecture.** Quel que soit l'entier de départ, la suite finit toujours par atteindre 1 (c'est la *conjecture de Syracuse*, encore non démontrée à ce jour).

Exercice 2 — L'escargot

► **1.** Le j -ième jour, l'escargot avance de $\frac{1}{j}$ mètre. On continue *tant que* la distance reste inférieure à 3 m ; à chaque tour on passe au jour suivant et on ajoute $\frac{1}{\text{jour}}$.

```
1 def parcours():
2     distance = 1
3     jour = 1
4     while distance < 3:
5         jour = jour + 1
6         distance = distance + 1/jour
7     return jour, distance
```

► La fonction renvoie (11, 3.0199...) : il faut **11 jours** pour que la distance dépasse 3 m (la distance totale vaut alors environ 3,02 m). C'est la somme $1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{11} \approx 3,02$.

► **2.** Il suffit de remplacer le seuil fixe 3 par un argument `seuil` :

```
1 def parcours(seuil):
2     distance = 1
3     jour = 1
4     while distance < seuil:
5         jour = jour + 1
6         distance = distance + 1/jour
7     return jour, distance
```

Exercice 3 — Fonction somme

- 1. Pour `somme(6)`, la variable `i` parcourt 0, 1, 2, 3, 4, 5, 6, et on n'ajoute `i` que lorsqu'il est pair : $0 + 2 + 4 + 6 = 12$.
- 2. La fonction renvoie la **somme des entiers pairs de 0 à n** .

Exercice 4 — Le chocolatier

- 1. Le coût de x chocolats est $C(x) = 200 + 0,15x$. On augmente x d'un chocolat *tant que* le coût reste ≤ 300 ; dès qu'il dépasse 300, on renvoie le dernier x valable, soit $x - 1$.

```

1 def cout():
2     C = 200
3     x = 0
4     while C <= 300:
5         x = x + 1
6         C = 200 + 0.15 * x
7     return x - 1

```

- La fonction renvoie **666** : avec 666 chocolats, $C = 200 + 0,15 \times 666 = 299,90 \text{ €}$ (≤ 300), alors qu'avec 667 chocolats $C = 300,05 \text{ €}$ (> 300).

- 2. On remplace 300 par un argument :

```

1 def cout(plafond):
2     C = 200
3     x = 0
4     while C <= plafond:
5         x = x + 1
6         C = 200 + 0.15 * x
7     return x - 1

```

Exercice 5 — Pyramide de cubes

- 1. Le k -ième cube a un côté de k cm, donc un volume de k^3 . Le volume total des n cubes est $1^3 + 2^3 + \dots + n^3$:

```

1 def cubes(n):
2     volume = 0
3     for k in range(1, n+1):
4         volume = volume + k**3
5     return volume

```

- 2. La fonction `nombre_cubes` augmente n *tant que* le volume total reste inférieur à V . Pour $V = 100\,000$:

$$\text{cubes}(24) = 90\,000 < 100\,000 \quad \text{et} \quad \text{cubes}(25) = 105\,625 \geq 100\,000,$$

donc `nombre_cubes(100000)` renvoie **25**.

- 3. Cela signifie qu'il faut empiler au minimum **25 cubes** (de côtés 1, 2, ..., 25 cm) pour que le volume total atteigne au moins $100\,000 \text{ cm}^3$ (soit 100 litres).

Exercice 6 — Fabrication de trousse

- 1. Chaque trousse est vendue 5 €, donc le chiffre d'affaires de x trousse est $G(x) = 5x$.
 ► 2. **Fonction B** (bénéfice = $G - C$) :

```
1 def B(x):
2     G = 5 * x
3     C = 0.000001*x**3 - 0.0002*x**2 + 0.1667*x
4     return G - C
```

- 3. **Fonction seuil**. On cherche le plus petit nombre de trousse pour lequel le bénéfice atteint 4 000 € : on augmente x tant que $B(x) < 4000$.

```
1 def seuil():
2     x = 0
3     while B(x) < 4000:
4         x = x + 1
5     return x
```

- La fonction renvoie **986** : $B(985) \approx 3999,2$ € (< 4000) tandis que $B(986) \approx 4001,5$ € (≥ 4000).

Exercice 7 — Vitesse d'un tsunami

- 1. **Fonction vitesse**.

```
1 from math import sqrt
2 def vitesse(h):
3     return 870 * sqrt(h/60)
```

- 2. Pour une profondeur de 3 km :

```
>>> vitesse(3)
194.5379...
```

soit une vitesse d'environ **194,5 km/h**.

- 3. **Fonction limite**. On fait croître la profondeur h par pas de 1 m ($\frac{1}{1000}$ km) tant que la vitesse reste inférieure à v_0 , puis on renvoie h . (La vitesse seuil v_0 doit être passée en argument.)

```
1 def limite(v0):
2     h = 0
3     while vitesse(h) < v0:
4         h = h + 1/1000
5     return h
```

- 4. Pour $v_0 = 900$: $\text{limite}(900)$ renvoie environ 64,210 km, soit une profondeur d'environ **64 210 m**. (On vérifie : $(\frac{900}{870})^2 \times 60 \approx 64,21$ km.)

Exercice 8 — Le chien

► **Idée.** La vitesse de départ est 9 m/s et elle est multipliée par 0,9 (diminution de 10 %) à chaque trajet. On compte les trajets *tant que* la vitesse reste ≥ 1 m/s.

```
1 def trajets():
2     vitesse = 9
3     n = 0
4     while vitesse >= 1:
5         n = n + 1
6         vitesse = vitesse * 0.9
7     return n
```

► La fonction renvoie **21**. En effet, la vitesse au trajet k vaut $9 \times 0,9^{k-1}$: au trajet 21 elle vaut $9 \times 0,9^{20} \approx 1,09$ m/s (≥ 1), et au trajet 22 elle vaut $\approx 0,98$ m/s (< 1). Le chien effectue donc **21 trajets** avant de s'arrêter.

► **Remarque.** La distance $A-B$ (100 m) n'intervient pas dans ce comptage : seule la vitesse décide de l'arrêt.

Exercice 9 — Les sargasses

► **1.** Augmenter de 4 % revient à multiplier par 1,04. On compte les semaines *tant que* la masse n'a pas dépassé 1 000 kg (la tonne).

```
1 def sargasses():
2     masse = 700
3     n = 0
4     while masse <= 1000:
5         masse = masse * 1.04
6         n = n + 1
7     return n
```

► **2.** La fonction renvoie **10** : après 9 semaines, $700 \times 1,04^9 \approx 996,3$ kg (≤ 1000) ; après 10 semaines, $700 \times 1,04^{10} \approx 1036,2$ kg (> 1000). La plage sera donc fermée dans **10 semaines**.

Exercice 10 — L'urne

► La boule rouge est numérotée 0 et les 99 noires de 1 à 99. On tire donc un entier au hasard entre 0 et 99 avec `randint(0, 99)`, et l'on continue *tant que* la boule n'est pas la rouge (c'est-à-dire *tant que* le tirage est $\neq 0$).

```
1 from random import randint
2 def boule():
3     nbre = 0
4     while randint(0, 99) != 0:
5         nbre = nbre + 1
6     return nbre
```

► **Ce que compte nbre.** La variable n'augmente que lorsque la boule tirée est noire : `nbre` compte donc les tirages *avant* d'obtenir la rouge. Le nombre total de tirages (boule rouge comprise) est `nbre + 1`.

Exercice 11 — Dé à 12 faces

Rappel de la fonction :

```
1 from random import randint
2 def douze():
3     resultat = "perdu"
4     n = 0
5     while randint(1, 12) != 12:
6         n = n + 1
7     return "nombre d'essais pour obtenir 12 : ", n
```

- 1. La fonction renvoie un couple, par exemple :

```
>>> douze()
('nombre d'essais pour obtenir 12 : ', 7)
```

Le nombre obtenu **varie à chaque appel** (en moyenne, il faut environ 12 lancers pour obtenir un 12).

- 2. Lignes 5 et 6.

- **Ligne 5** : `while randint(1, 12) != 12` — on relance le dé à 12 faces *tant que* le résultat n'est pas 12.
- **Ligne 6** : `n = n + 1` — on compte chaque lancer qui n'a *pas* donné 12.

- **Remarque.** La variable `resultat = "perdu"` (ligne 3) n'est jamais utilisée par la suite.